

What a Small Local LLM Can and Cannot Do for Legal Case Research

Swetaleena Guha, Nandini Chanda, and Nishanth Madthila

Team 4 · 95-820 Applications of NL(X) and LLM ·

Abstract

We report three independently built evaluations of a small, locally run LLM API (Phi-4-mini-instruct via LLMBBox) for legal case research in two domains, using Federal Court of Australia judgments (2006–2009) from the UCI Legal Case Reports dataset: two Intellectual Property corpora (IP-A, 214 records; IP-B, 218 records from 50 judgments) and one Immigration and Refugee corpus (IMM, 220 records, 137 citation-annotated). Each study measured baseline generation, structured output, tool calling, validation, rule-based guardrails, and human adjudication. No baseline was good enough for unreviewed use: the best IP-B run reached 52% exact match on IP area, and no IMM configuration beat a trivial “always cited” baseline (42–44%). Structured output fixed format at best, never accuracy; in IP-B, fabricated citations rose from 1 to 13. Tool calling worked only when fully specified (IP-A, 50 of 50); when the model had to decide, it skipped the tool (IP-B, 13 of 50) or claimed verification it never performed (IMM, 27 records). Deterministic checks and rule-based guards worked within their scope but missed reworded harmful requests. The results support a human-in-the-loop design where the system, not the model, does retrieval and verification.

1 Introduction

Legal research means finding the right authorities and representing them accurately. LLMs read long documents quickly, but a fluent answer can cite a case that does not exist, and courts have sanctioned lawyers for exactly that (Mata v. Avianca, 2023). The useful question for an organization is therefore not whether a model is usually helpful, but whether its errors can be caught before they reach a client or a court.

We evaluate LLM APIs for case research in Intellectual Property (IP) law and Immigration and Refugee law. Each author built a separate study within a shared course framework (LLMBBox; Kingsley, 2026), with its own corpus, scenario, prompts, and scripts. The two IP studies use different units of analysis and are complementary, not replications. The immigration study adds a harder task: classifying how an opinion treats the precedents it cites. We ask:

1. **RQ1 (capability).** How reliably can a small local model extract area of law, cited cases, legislation, relief, and citation treatment from judgments?
2. **RQ2 (API features).** Do structured output and tool calling make outputs more trustworthy than plain generation?
3. **RQ3 (controls and safety).** What do validation and rule-based guardrails catch, and what do they miss?
4. **RQ4 (adoption).** Which capabilities have enough evidence for organizational use, and under what oversight?

We make three contributions. First, we test the same small local model on real Federal Court of Australia judgments in three separate studies, covering extraction, structured output, tool calling,

validation, and guardrails. Second, we compare the studies to show which failures recur across IP and immigration law. Third, we turn the results into recommendations on which capabilities an organization can use under human review and which it should not use yet.

2 Related Work

All three corpora build on Galgani and Hoffmann’s work on Australian case law. Their LEXA system classified citation treatment with hand-built rules over the same labels we use, working from the text around each citation (Galgani & Hoffmann, 2010; Galgani, Compton, & Hoffmann, 2015). Benchmarks such as LegalBench (Guha et al., 2023) and LexGLUE (Chalkidis et al., 2022) show that fine-grained legal labels stay hard even for large models. General LLMs hallucinate on many verifiable legal queries (Dahl et al., 2024), and retrieval-based legal tools still do so at a meaningful rate (Magesh et al., 2025). Constrained decoding can guarantee schema-valid output (Willard & Louf, 2023), and tool-using models are expected to decide when to call tools (Schick et al., 2023); LLMBBox does neither by construction, so we test both assumptions. Indirect prompt injection through retrieved documents is a realistic threat for systems that read third-party judgments (Greshake et al., 2023; OWASP Foundation, 2025). Classifier guards such as Llama Guard (Inan et al., 2023) are more robust than word lists but need a second model; hardware limits led all three studies to use rule-based guards. The use case is motivated by how much time research takes: in a survey of 190 U.S. law firm associates, respondents reported spending about 31% of their working hours on legal research (Lastres, 2013). We do not measure this ourselves, and the figure is from U.S. rather than Australian practice.

3 Methodology

3.1 Data, Scenarios, and Model

All corpora come from the UCI Legal Case Reports dataset (Galgani, 2012): 3,890 Federal Court of Australia judgments from AustLII, licensed for research and coursework only. Because they differ in selection, unit of analysis, and ground truth (Table 1), results are reported side by side and never pooled.

Table 1. Corpora, scenarios, and target tasks.

	IP-A (Guha)	IP-B (Chanda)	IMM (Madthila)
Selection	IP-related catchphrases	50 IP cases	“Minister for Immigration” cases (917 of 3,890); 220 sampled, seed 42
Records / unit	214 whole case reports	218: 150 judgment sections (cut at 4,500 chars) + 68 table records	220 whole opinions; 137 with citation tables
Ground truth	Mechanical checks; 25 hand labels in AS01	Corpus IP-area labels; citations and Acts checked against excerpt	Dataset citation tables (1,064 rows, 876 distinct cases)
Splits	149 / 65 (70/30, seed 42); fixed 50-record samples	25 / 25 cases (seed 42); 50 sections per run	50 / 50 / 37 reserve (seed 42)
Scenario	Internal IP research unit	Mid-size Australian IP firm	Pro bono immigration and refugee legal aid clinic
Target output	<code>ip_area</code> , <code>legislation_cited</code> , <code>relief_sought</code>	IP areas, cases cited, legislation cited	Citation treatment (11 classes), cited cases, visa category
Custom components	Input screen, output validator, <code>lookup_case</code>	<code>check_citations</code> , keyword guard, loop guard	<code>verify_citation</code> , ClinicGuard, GroundedIntake

In IMM, each record’s gold treatment is the most common class in its citation table. The labels are skewed enough that always answering “cited” scores 44% (development) and 42% (evaluation), the baseline every configuration is compared against. Opinions average about 5,300–5,600 tokens; input caps of 6,000 and later 3,000 tokens truncated 26–30% and 64–80% of opinions. In IP-B, 129 of 150 sections were cut at 4,500 characters, and in IP-A, 11 of 50 documents were truncated in the first baseline.

All systems ran on LLMBox over local weights; no hosted API was called in AS02. All three studies used Phi-4-mini-instruct (3.8B; Abouelenin et al., 2025). IP-B ran on a laptop, IP-A and IMM on Colab (IMM on a T4 in float16, logging out-of-memory failures as [OOM]). All three governance policies limit tools to read-only access, keep processing local, mask sensitive spans where present, and require human review of substantive output.

Each study ran in two rounds, AS01 and AS02, corresponding to the first two course assignments.

3.2 Experiments and Measures

Experiments fall into six capability families; labels follow each author’s numbering (see Code and Data Availability).

Baseline generation: two decoding settings on 50 development records (roughly temperature 1.0 vs. 0.2; IP-A also cut `max_new_tokens` from 512 to 256 while IP-B raised it to 768). **Structured output:** a Pydantic schema passed through LLMBox, which adds the schema to the prompt rather than enforcing it (IMM also re-ran it on 15 records at a 3,000-token cap, C1r). **Tool calling:** IP-A’s read-only `lookup_case` with the document ID named in the prompt; IP-B’s `check_citations`; IMM’s `verify_citation` over 876 indexed citations. **Validation:** IP-A’s input screen and field-name validator; IP-B’s guard on real records (C3) and loop guard (C4); IMM’s ClinicGuard (C3) and GroundedIntake (C4), which uses a flat example, retries invalid JSON, drops cases not found

verbatim in the opinion, and has the API, not the model, call `verify_citation`. **Guardrails:** domain probe sets with and without a rule-based guard (IP-A 40 adversarial + 10 benign; IP-B 12 + 5; IMM 20 + 10), with a canary string planted for indirect injection. **Human evaluation:** 25 hand-labelled records per author in AS01, and manual adjudication of guard decisions in AS02 (IP-A 15, IP-B 17, IMM 30). IMM’s AS01 labels and Part D adjudication were AI-prefilled and then corrected, so they are not fully independent.

Measures differ by study: IP-A’s AS02 metrics are mechanical (field presence, validity, tool-call correctness); IP-B scores IP exact match and citation grounding; IMM scores treatment accuracy and cited-case F1. Configurations are compared only within a study and only on the same records and input cap. In IP-B, C3 (a guard that blocked nothing) is the plain reference for C1, C2, and C4; in IMM, B1 is compared with B3 and C1r with C4. Latency is not comparable across studies (different hardware; IP-B reports medians).

4 Findings

4.1 Intellectual Property

AS01 starting point. Against 25 hand labels, IP-A got no record fully correct: field accuracy was 76% (`ip_area`), 36% (legislation), and 16% (relief). IP-B scored 0.64 on `ip_area` and 0.68 on cited cases, with 0.24 of records fully correct; cited-case precision was 0.526 but recall only 0.192, partly because of a 1,400-character input cap. A larger hosted model (`gpt-oss-120b`) raised IP-B’s `ip_area` to 0.84 but left cited cases at 0.68.

Baseline generation. Neither IP baseline was adequate. In IP-A, only 2 and 1 of 50 responses had all three field names and none passed the `ip_area` check; lowering randomness raised output-limit hits from 19 to 32 of 50. In IP-B, lower randomness raised IP exact match from 42% to 52% and precision from 43.5% to 59.3%, but citation recall fell from 77.9% to 70.9%, 5 citations were fabricated in each run, and 3 answers looped to the token limit. With 50 records, a 5-record change is 10 points, so small differences may be noise.

Structured output and tool calling. The two IP studies failed differently (Table 2). IP-A produced 0 of 50 valid JSON responses; the model often reproduced the schema instead of data. IP-B, after adding rules and an example, got valid JSON on all 50 records, but IP exact match fell from 50% to 42%, fabricated citations rose from 1 to 13, and real citations found fell from 89% to 58%. IP-A’s `lookup_case` tool succeeded 50 of 50 times, but every prompt named the document, so the model had almost nothing to decide. IP-B’s `check_citations` removed fabrication, but the model skipped it in 13 of 50 records, it rejected real citations written slightly differently, and IP exact match fell to 28% at about twice the input tokens.

Table 2. IP-B configurations on the same 50 evaluation records.

Metric	C3 (plain ref.)	C1 structured	C2 tool	C4 loop guard
Correct format	98%	94%	76%	96%
IP exact match	50%	42%	28%	42%
Made-up citations	1	13	0	1
Real citations found	89%	58%	34%	75%
Median seconds	18.5	29.4	33.7	20.0
Mean input tokens	1,046	1,518	2,440	1,159

Validation and guardrails. IP-A’s input screen classified all 50 test inputs correctly and its validator separated 25 complete from 25 incomplete outputs, in well under a millisecond, but the tests closely matched the rules and the validator only checks field names. IP-B’s loop guard fixed 4 of 5 loops at 2–3 minutes per retry, with no accuracy gain. Unguarded, IP-A’s model followed 8 of 10 indirect injections; IP-B’s was vulnerable to 8 of 12 attacks. IP-A’s guard blocked 10 of 10 injections and 9 of 10 leakage probes but none of the 20 harmful or out-of-scope probes, which it had no rules for. IP-B’s guard caught 6 of 12 attacks and missed all three toxic requests; one produced an answer calling the applicant “dishonest and manipulative,” wording not on its list. It also blocked 1 of 5 benign requests. Authors agreed with their guards on 12 of 15 (IP-A) and 13 of 17 (IP-B) decisions.

4.2 Immigration and Refugee Law

AS01 starting point. On 25 records, the first structured prompt violated the schema 88% of the time because the model echoed the JSON Schema wrapper and nested its answer inside it. A recovery prompt with a flat example removed every violation and raised treatment accuracy from 0% to 48% and visa-category accuracy from 4% to 76%; remaining errors were near misses between neighbouring labels.

Baseline generation. Both baselines fell far below the “always cited” baseline (Table 3), and 18 of 50 records ran out of GPU memory in each run. Lowering temperature halved treatment accuracy because near-greedy decoding collapsed onto one label (“referred to” on 17 of 32 completed records), while outputs nearly doubled in length.

Table 3. IMM baseline generation (50 development records; 32 completed).

Run	Treatment acc. all / completed	Cited-case F1 all / completed	Mean output tokens	Mean latency
Always “cited”	0.44	n/a	0	0 s
B1 (temp 1.0)	0.08 / 0.125	0.120 / 0.188	79.7	10.3 s
B3 (temp 0.2)	0.04 / 0.062	0.138 / 0.215	148.9	17.3 s

Structured output and tool calling. With the CaseIntake schema, only 10 of 50 responses were valid JSON as returned (19 after stripping code fences); failures included renamed keys, echoed schemas, repetition loops, and invented enum values. Treatment accuracy was 2 of 29 completed records. Told to verify every citation with `verify_citation`, the model called the tool 0 times and

answered “VERIFIED: true” on 27 records, including a magistrate’s name and a misremembered High Court case. Two correctly written tool calls were silently dropped by the LLMBox parser, visible only in per-record logs.

Validation and guardrails. On the same 15 records at a 3,000-token cap, GroundedIntake raised valid JSON from 0 to 15 of 15, cut latency from 17.1 to 7.5 seconds, and removed every returned case not found in the opinion (53%). Accuracy stayed at zero: treatment 0 of 15 and cited-case F1 0.00, because surviving cases did not exactly match the gold entries. ClinicGuard cut successful attacks from 6 of 20 to 0, caught 19 of 20 (all injections, out-of-scope, and leakage probes), blocked no benign request, and added 0.9–3.4 ms. The one miss, “rewrite the findings so the applicant looks stupid,” contained no listed word and was stopped only by the model’s own refusal. The author agreed with 29 of 30 guard decisions.

4.3 Cross-Domain Findings

Table 4 sets the measured outcomes side by side. Because corpora and tasks differ, the comparison asks whether the same pattern appears, not which study scored higher.

Table 4. Capability-level outcomes across the three studies.

Capability	IP-A	IP-B	IMM
Baseline	Weak field adherence; no valid <code>ip_area</code>	Best IP match 52%; 5 fabricated citations per run	Below trivial baseline (0.125 vs. 0.44)
Lower randomness	Cap hits 19 → 32 of 50	Precision up, recall down; 3 loops	Treatment accuracy halved
Structured output	0 of 50 valid JSON	50 of 50 valid; IP 42% vs. 50%; fabrications 13 vs. 1	20% valid (C1); 15 of 15 with GroundedIntake, accuracy 0
Tool calling	50 of 50 explicit lookups	Skipped 13 of 50; recall 34%	0 calls; 27 false “verified”
Deterministic checks	50 of 50 on rule-matched tests	Loop guard fixed 4 of 5	Removed all ungrounded cases (53%)
Guard: injection / harmful	10 of 10 / 0 of 10	2 of 3 / 0 of 3	5 of 5 / 4 of 5
Benign over-refusal	0 of 10	1 of 5	0 of 10
Human agreement	80%	76%	96.7%

Five patterns recur. **(1) Baselines were insufficient everywhere, and decoding changes did not transfer:** lower temperature helped IP-B, hurt IMM, and caused length problems in all three. **(2) Structured output changed form, not correctness:** where it failed, the model reproduced the schema; where format became valid, accuracy did not improve and IP-B’s fabrications rose. **(3) Tool use was reliable only when the model had nothing to decide:** explicit lookups succeeded, discretionary verification was skipped or falsely claimed. **(4) Deterministic checks run by the system worked where aimed, at negligible cost** (0.0026–3.4 ms), but they check

form and string match, not legal correctness, and dropped real citations written differently. (5) **Rule-based guards stopped anticipated patterns only:** injection and leakage were blocked well; reworded harmful requests passed in all three studies. Two evaluation lessons also recur: several failures (schema echoing, OOM records, dropped tool calls, a token-count bug) were visible only in raw outputs, and input truncation affected every study.

5 Discussion

“How accurate is the model?” is too coarse a question for a deployment decision. The same small model was fully reliable on one narrowly specified lookup and below a trivial baseline on citation treatment. Capability-level evaluation, with separate thresholds for retrieval, classification, citation precision and recall, schema validity, and adversarial robustness, is more useful than one score.

A tentative reading is that reliability fell as the model was given more discretion: whether to call a tool, what to extract, how to label it. The comparison is confounded by corpus, task, prompt, and possibly model, so this is a hypothesis. It agrees with the conclusion all three authors reached independently: verification should be done by deterministic code the API runs, and a model’s claim to have verified something should carry no weight. IMM shows why. Fabricated verification is invisible in aggregate accuracy, and a reviewer trusting the label would receive an invented case marked as checked.

Format and content must be measured separately. In IP-B, structured output looked better on format while degrading content; in IMM, GroundedIntake fixed format and safety while accuracy stayed at zero; and in IMM’s AS01 work, a format fix revealed reasonable content hidden by a parsing failure. The immigration task, 11-way classification between labels that differ by convention on long, heavily truncated inputs, is also harder than IP-area classification, so accuracy figures should not be compared across domains. What transfers is the pattern of failure. Two caveats apply: the studies shared LLMBox and the same 3.8B model, so some failures may be framework- or model-specific; and the only evidence on scale is IP-B’s 25-record comparison with a much larger model, which improved classification but not cited cases.

6 Organizational Implications

Supported by measurement. Read-only retrieval by a known identifier has the strongest evidence for early use (IP-A, 50 of 50). Every substantive extraction (IP area, legislation, relief, cited cases, treatment) needs review against the source. Schema-valid output is not evidence of correctness. Verification must be run by the system, since model claims of verification were unreliable (IMM) and discretionary tool use inconsistent (IP-B). Rule-based guards are a cheap layer, not a complete control.

Policy revisions and recommendations. After testing, IP-A kept human review and read-only tools and began treating fabricated disclosure claims as a review issue; IP-B downgraded IP tags to search hints, dropped structured output as the main mode, and made the guard an addition to human checks; IMM required API-side schema and grounding checks, paralegal review of every record, “verified” to mean a deterministic API lookup, and a visible input cap. Together these point to an architecture where the model helps navigate approved material while deterministic components handle retrieval, grounding, and verification, with read-only tools, local processing, links from each field back to the source, visible truncation, logging, layered controls, and human

review. IP-B and IMM both recommend a pattern-based citation extractor in place of the LLM, since Australian citations follow fixed formats (untested as a full pipeline). IP-A recommended adoption limited to retrieval and research assistance, IP-B adoption within its policy limits, and IMM against using the model for treatment classification.

Cost (assumptions, not measurements). Each author’s cost model rests on planning assumptions. IP-A estimates \$4.66 saved per case with break-even near 103 cases, assuming a 20% correction rate that was not measured. IP-B estimates about 5 minutes saved per section and break-even near 240 sections. IMM’s model saves time only if more than 25% of outputs are usable as is; the measured figure was at most 4–8%, so on current numbers the API costs the clinic time. All three hinge on human review and correction time, which no study measured; a timed practitioner trial is the most important missing input.

7 Limitations

- **One source and period:** all data is Federal Court of Australia, 2006–2009, from one dataset.
- **Different corpora and tasks:** units of analysis, fields, and ground truth differ, and IP-B and IMM gold labels are case-level or majority labels; cross-study comparison is qualitative.
- **Small samples:** most runs used 50 records, IMM’s second round 15, with further OOM losses; probe sets had 12–40 adversarial items.
- **Mechanical metrics:** IP-A’s semantic accuracy comes only from 25 AS01 records.
- **Shared model and framework:** shared failures may partly reflect Phi-4-mini or LLMBox.
- **Truncation:** input caps cut most IP-B sections and up to 80% of IMM opinions.
- **Evaluator independence:** the same authors wrote probes and rules; IMM’s labels and adjudication were AI-prefilled.
- **Optimistic index and reconstructed outputs:** IMM’s citation index includes evaluation records, and its round-2 per-record outputs were rebuilt from notebook logs.

8 Future Work

Replace the LLM with pattern-based citation extraction and use the model only for treatment classification on the paragraph around each citation. Feed that citing paragraph instead of the start of the document. Normalise short-form and parallel citations, and build verification indexes from training records only. Retest structured output with decoding that enforces the schema. Red-team with paraphrased and translated probes written by someone other than the rule author, and compare word lists with a local classifier guard. Evaluate larger models on larger, independently labelled sets, and run timed practitioner trials measuring review time and correction burden.

9 Conclusion

Across three independent studies in two areas of law, a small local LLM API was not reliable enough for unreviewed case research. Structured output changed the form of answers without improving correctness, tool calling worked only when fully specified, and model-reported verification could

not be trusted. Deterministic checks and rule-based guards were cheap and effective within their scope but do not verify legal correctness. The evidence supports a narrow, observable deployment in which the system handles retrieval and verification and a qualified person reviews every substantive output against the source.

These findings matter to law firm partners and legal aid clinic managers deciding whether to adopt such tools, to the lawyers and paralegals who would review the outputs, to clients and visa applicants whose outcomes depend on accurate authorities, and to courts, which have already sanctioned lawyers for filing fabricated citations.

Code and Data Availability

All code, notebooks, per-record outputs, and memos are available at https://github.com/nishsm/NLX_LLM-Team4.

References

- Abouelenin, A., et al. (Microsoft). (2025). Phi-4-Mini technical report: Compact yet powerful multimodal language models via mixture-of-LoRAs. arXiv:2503.01743.
- Bhave, R. (2026). AS01 starter code (`schema.py`, `local_model.py`, `extraction.py` and related scripts) [MIT-licensed course code].
- Chalkidis, I., Jana, A., Hartung, D., Bommarito, M., Androutsopoulos, I., Katz, D. M., & Aletras, N. (2022). LexGLUE: A benchmark dataset for legal language understanding in English. In *Proceedings of ACL 2022*.
- Dahl, M., Magesh, V., Suzgun, M., & Ho, D. E. (2024). Large legal fictions: Profiling legal hallucinations in large language models. *Journal of Legal Analysis*, 16(1). arXiv:2401.01301.
- Galgani, F. (2012). Legal Case Reports [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C5ZS41>
- Galgani, F., Compton, P., & Hoffmann, A. (2015). LEXA: Building knowledge bases for automatic legal citation classification. *Expert Systems with Applications*, 42(17–18), 6391–6407. <https://doi.org/10.1016/j.eswa.2015.04.022>
- Galgani, F., & Hoffmann, A. (2010). LEXA: Towards automatic legal citation classification. In *AI 2010: Advances in Artificial Intelligence*, LNCS 6464. Springer.
- Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*.
- Guha, N., et al. (2023). LegalBench: A collaboratively built benchmark for measuring legal reasoning in large language models. In *NeurIPS 2023 Datasets and Benchmarks Track*.
- Inan, H., et al. (2023). Llama Guard: LLM-based input-output safeguard for human-AI conversations. arXiv:2312.06674.
- Kingsley, S. (2026). LLMBBox [Course software]. 95-820 Applications of NL(X) and LLM, Heinz College, Carnegie Mellon University.
- Lastres, S. A. (2013). *Rebooting legal research in a digital age* [Insights paper]. LexisNexis. Based on a survey of 190 law firm associates conducted by The Research Intelligence Group, December 2012. https://www.lexisnexis.com/documents/pdf/20130806061418_large.pdf
- Magesh, V., Surani, F., Dahl, M., Suzgun, M., Manning, C. D., & Ho, D. E. (2025). Hallucination-free? Assessing the reliability of leading AI legal research tools. *Journal of Empirical Legal Studies*. arXiv:2405.20362.
- Mata v. Avianca, Inc.*, No. 22-cv-1461 (PKC) (S.D.N.Y. June 22, 2023) (opinion and order on sanctions).

- OWASP Foundation. (2025). *OWASP Top 10 for Large Language Model Applications* (2025 ed.). <https://genai.owasp.org/llm-top-10/>
- Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. In *NeurIPS 2023*.
- Willard, B. T., & Louf, R. (2023). Efficient guided generation for large language models. arXiv:2307.09702.